

# **Computer Architecture**

**Lecture 26-27**

# Introduction

- Goal: connecting multiple computers to get higher performance
  - Multiprocessors
  - Scalability, availability, power efficiency
- Task-level (process-level) parallelism
  - High throughput for independent jobs
- Parallel processing program
  - Single program run on multiple processors
- Multicore microprocessors
  - Chips with multiple processors (cores)

# Hardware and Software

- Hardware
  - Serial: e.g., Pentium 4
  - Parallel: e.g., quad-core Xeon e5345
- Software
  - Sequential: e.g., matrix multiplication
  - Concurrent: e.g., operating system
- Sequential/concurrent software can run on serial/parallel hardware
  - Challenge: making effective use of parallel hardware

# Parallel Programming

- Parallel software is the problem
- Need to get significant performance improvement
  - Otherwise, just use a faster uniprocessor, since it's easier!
- Difficulties
  - Partitioning
  - Coordination
  - Communications overhead

# Amdahl's Law

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
  - $S$  is serial portion
  - $N$  processing cores

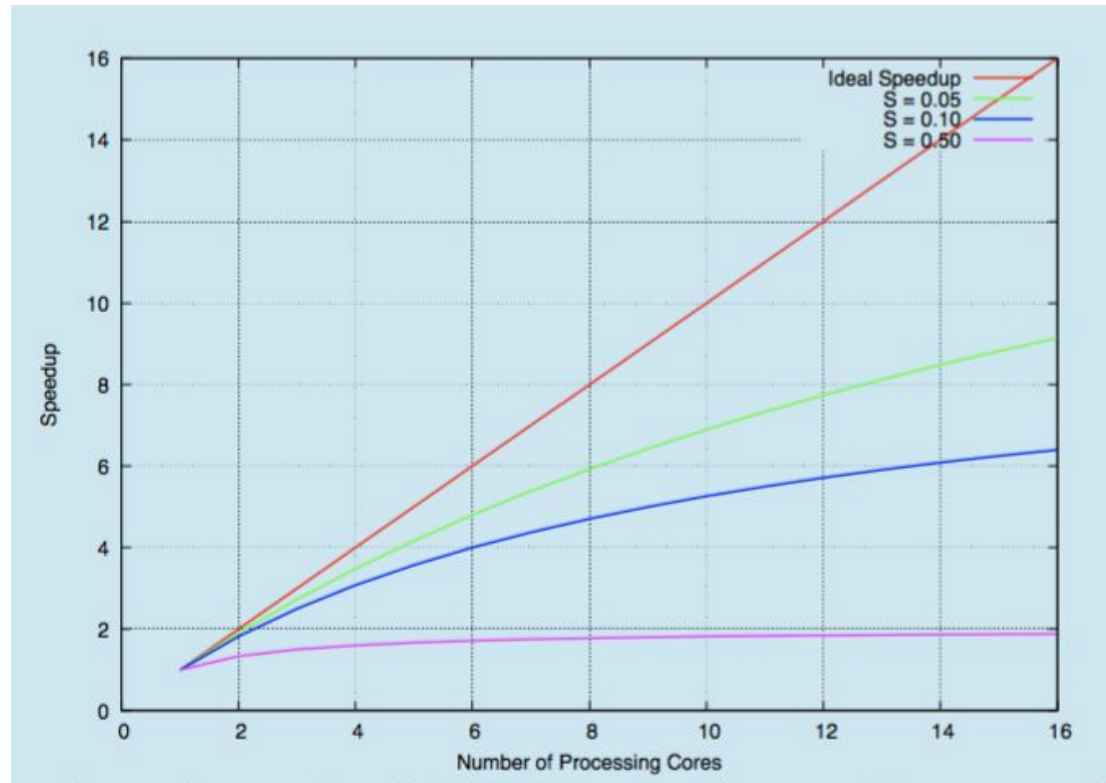
$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As  $N$  approaches infinity, speedup approaches  $1 / S$

**Serial portion of an application has disproportionate effect on performance gained by adding additional cores**

- But does the law take into account contemporary multicore systems?

# Amdahl's Law



# Instruction and Data Streams

## ■ An alternate classification

|                     |          | Data Streams                      |                                         |
|---------------------|----------|-----------------------------------|-----------------------------------------|
|                     |          | Single                            | Multiple                                |
| Instruction Streams | Single   | <b>SISD:</b><br>Intel Pentium 4   | <b>SIMD:</b> SSE<br>instructions of x86 |
|                     | Multiple | <b>MISD:</b><br>No examples today | <b>MIMD:</b><br>Intel Xeon e5345        |

- SPMD: Single Program Multiple Data
  - A parallel program on a MIMD computer
  - Conditional code for different processors

# Vector Processors

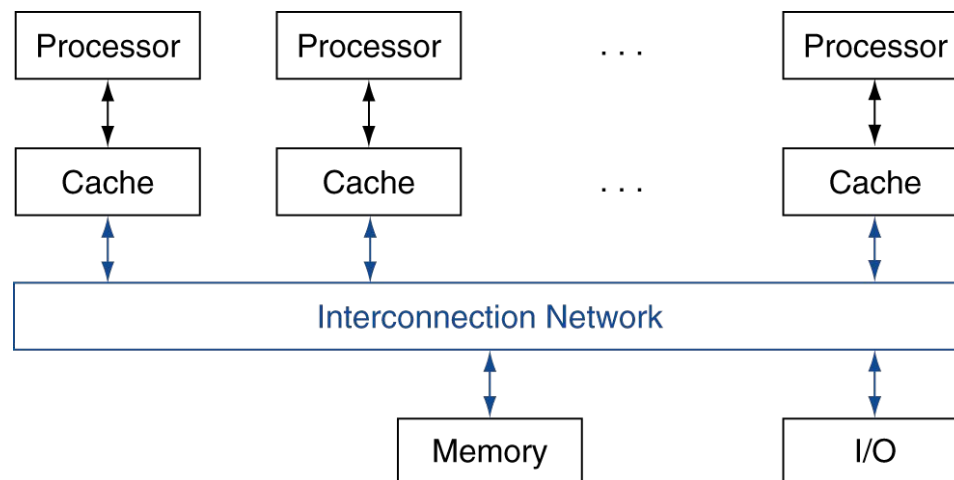
- Highly pipelined function units
- Stream data from/to vector registers to units
  - Data collected from memory into registers
  - Results stored from registers to memory
- Example: Vector extension to RISC-V
  - v0 to v31:  $32 \times 64$ -element registers, (64-bit elements)
  - Vector instructions
    - `fld.v`, `fsd.v`: load/store vector
    - `fadd.d.v`: add vectors of double
    - `fadd.d.vs`: add scalar to each element of vector of double
- Significantly reduces instruction-fetch bandwidth

# SIMD

- Operate elementwise on vectors of data
  - E.g., MMX and SSE instructions in x86
    - Multiple data elements in 128-bit wide registers
- All processors execute the same instruction at the same time
  - Each with different data address, etc.
- Simplifies synchronization
- Reduced instruction control hardware
- Works best for highly data-parallel applications

# Shared Memory

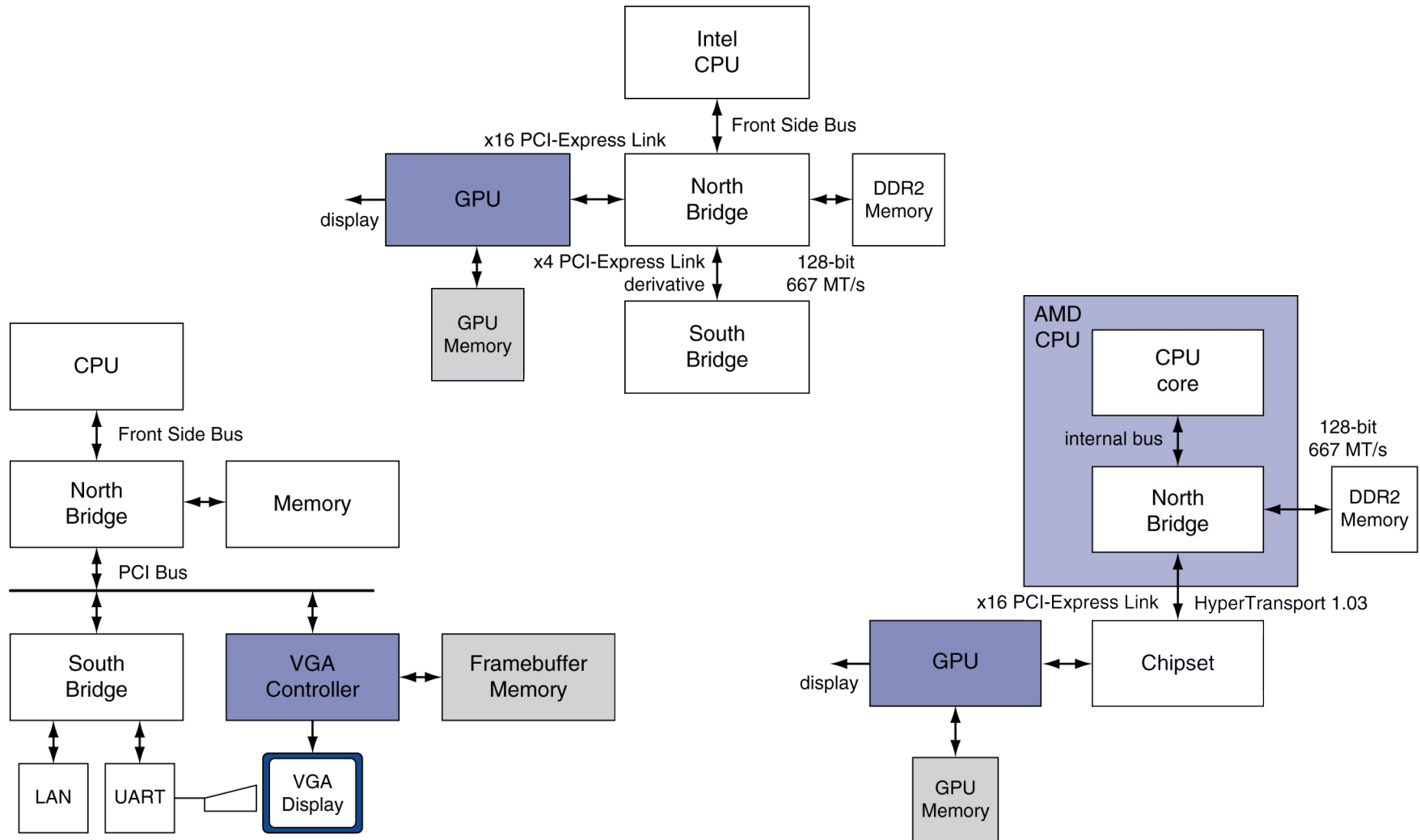
- SMP: shared memory multiprocessor
  - Hardware provides single physical address space for all processors
  - Synchronize shared variables using locks
  - Memory access time
    - UMA (uniform) vs. NUMA (nonuniform)



# History of GPUs

- Early video cards
  - Frame buffer memory with address generation for video output
- 3D graphics processing
  - Originally high-end computers (e.g., SGI)
  - Moore's Law  $\Rightarrow$  lower cost, higher density
  - 3D graphics cards for PCs and game consoles
- Graphics Processing Units
  - Processors oriented to 3D graphics tasks
  - Vertex/pixel processing, shading, texture mapping, rasterization

# Graphics in the System

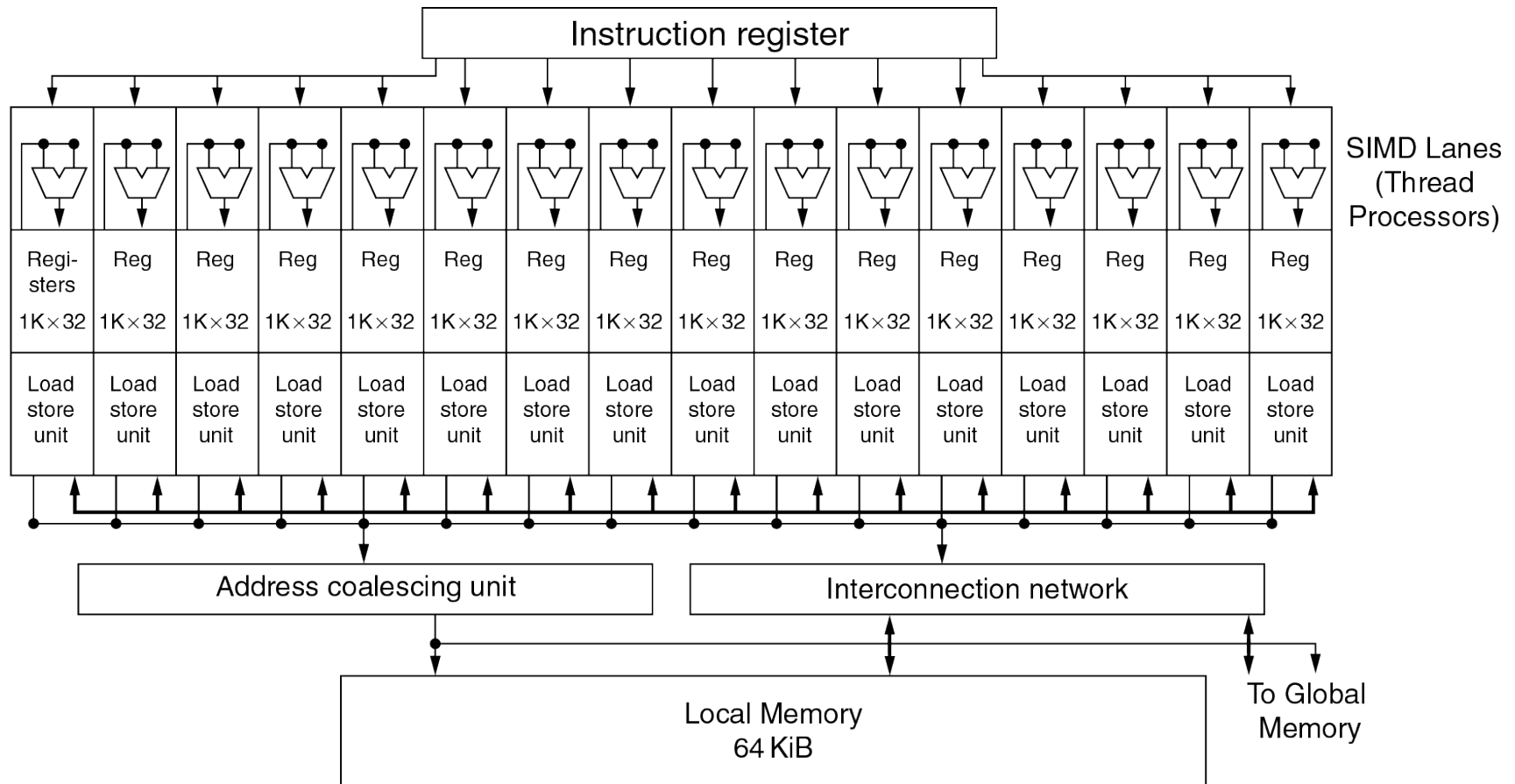


# GPU Architectures

- Processing is highly data-parallel
  - GPUs are highly multithreaded
  - Use thread switching to hide memory latency
    - Less reliance on multi-level caches
  - Graphics memory is wide and high-bandwidth
- Trend toward general purpose GPUs
  - Heterogeneous CPU/GPU systems
  - CPU for sequential code, GPU for parallel code
- Programming languages/APIs
  - DirectX, OpenGL
  - C for Graphics (Cg), High Level Shader Language (HLSL)
  - Compute Unified Device Architecture (CUDA)

# Example: NVIDIA Fermi

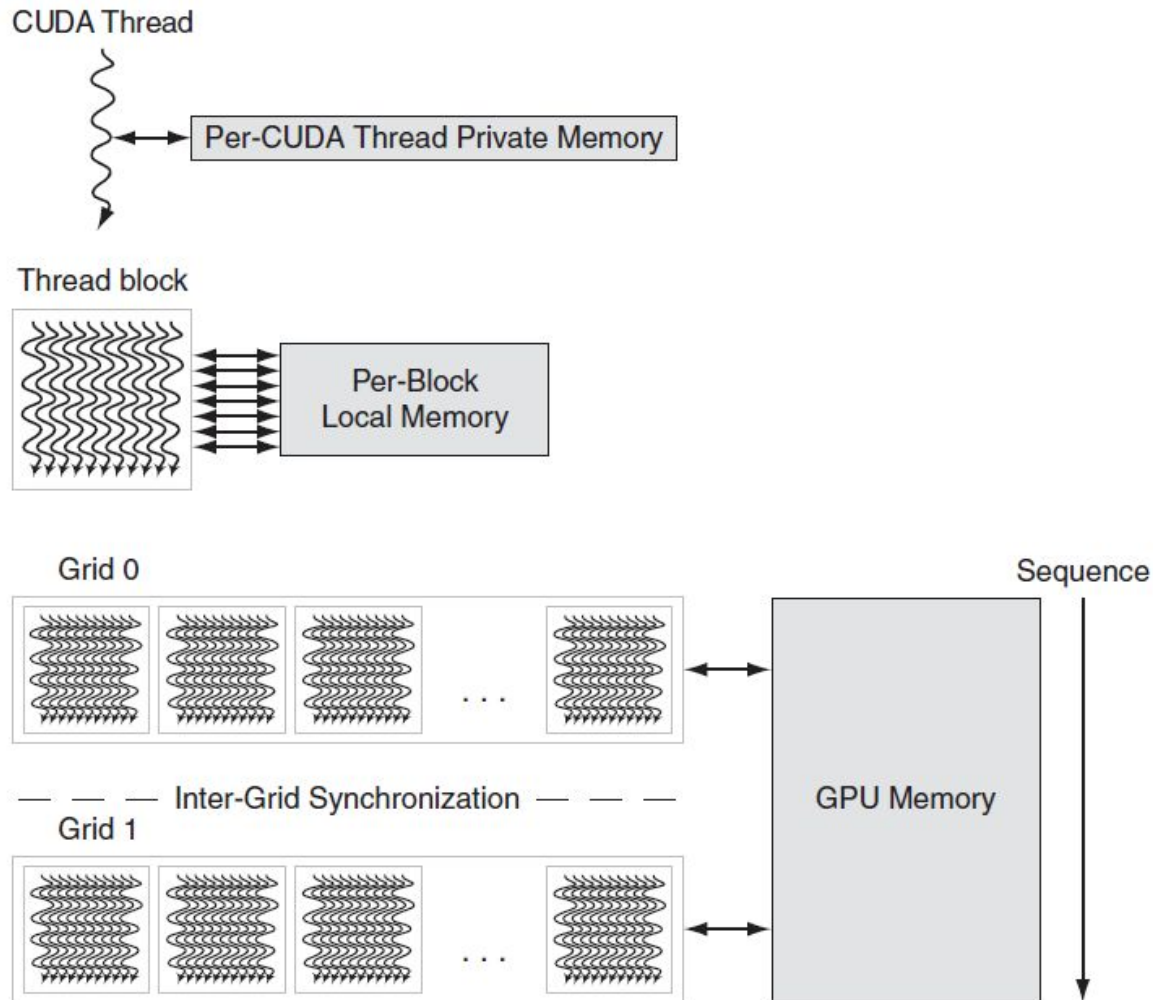
- Multiple SIMD processors, each as shown:



# Example: NVIDIA Fermi

- SIMD Processor: 16 SIMD lanes
- SIMD instruction
  - Operates on 32 element wide threads
  - Dynamically scheduled on 16-wide processor over 2 cycles
- 32K x 32-bit registers spread across lanes
  - 64 registers per thread context

# GPU Memory Structures



# Classifying GPUs

- Don't fit nicely into SIMD/MIMD model
  - Conditional execution in a thread allows an illusion of MIMD
    - But with performance degradation
    - Need to write general purpose code with care

|                                  | Static: Discovered<br>at Compile Time | Dynamic: Discovered<br>at Runtime |
|----------------------------------|---------------------------------------|-----------------------------------|
| Instruction-Level<br>Parallelism | VLIW                                  | Superscalar                       |
| Data-Level<br>Parallelism        | SIMD or Vector                        | <b>Tesla Multiprocessor</b>       |

# Guide to GPU Terms

| Type                 | More descriptive name            | Closest old term outside of GPUs        | Official CUDA/NVIDIA GPU term | Book definition                                                                                                                                                    |
|----------------------|----------------------------------|-----------------------------------------|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Program abstractions | Vectorizable Loop                | Vectorizable Loop                       | Grid                          | A vectorizable loop, executed on the GPU, made up of one or more Thread Blocks (bodies of vectorized loop) that can execute in parallel.                           |
|                      | Body of Vectorized Loop          | Body of a (Strip-Mined) Vectorized Loop | Thread Block                  | A vectorized loop executed on a multithreaded SIMD Processor, made up of one or more threads of SIMD instructions. They can communicate via Local Memory.          |
|                      | Sequence of SIMD Lane Operations | One iteration of a Scalar Loop          | CUDA Thread                   | A vertical cut of a thread of SIMD instructions corresponding to one element executed by one SIMD Lane. Result is stored depending on mask and predicate register. |
| Machine object       | A Thread of SIMD Instructions    | Thread of Vector Instructions           | Warp                          | A traditional thread, but it contains just SIMD instructions that are executed on a multithreaded SIMD Processor. Results stored depending on a per-element mask.  |
|                      | SIMD Instruction                 | Vector Instruction                      | PTX Instruction               | A single SIMD instruction executed across SIMD Lanes.                                                                                                              |
| Processing hardware  | Multithreaded SIMD Processor     | (Multithreaded) Vector Processor        | Streaming Multiprocessor      | A multithreaded SIMD Processor executes threads of SIMD instructions, independent of other SIMD Processors.                                                        |
|                      | Thread Block Scheduler           | Scalar Processor                        | Giga Thread Engine            | Assigns multiple Thread Blocks (bodies of vectorized loop) to multithreaded SIMD Processors.                                                                       |
|                      | SIMD Thread Scheduler            | Thread scheduler in a Multithreaded CPU | Warp Scheduler                | Hardware unit that schedules and issues threads of SIMD instructions when they are ready to execute; includes a scoreboard to track SIMD Thread execution.         |
|                      | SIMD Lane                        | Vector lane                             | Thread Processor              | A SIMD Lane executes the operations in a thread of SIMD instructions on a single element. Results stored depending on mask.                                        |
| Memory hardware      | GPU Memory                       | Main Memory                             | Global Memory                 | DRAM memory accessible by all multithreaded SIMD Processors in a GPU.                                                                                              |
|                      | Local Memory                     | Local Memory                            | Shared Memory                 | Fast local SRAM for one multithreaded SIMD Processor, unavailable to other SIMD Processors.                                                                        |
|                      | SIMD Lane Registers              | Vector Lane Registers                   | Thread Processor Registers    | Registers in a single SIMD Lane allocated across a full thread block (body of vectorized loop).                                                                    |

# Concluding Remarks

- Goal: higher performance by using multiple processors
- Difficulties
  - Developing parallel software
  - Devising appropriate architectures
- SaaS importance is growing and clusters are a good match
- Performance per dollar and performance per Joule drive both mobile and WSC

# Concluding Remarks (con't)

- SIMD and vector operations match multimedia applications and are easy to program

